

Rider Acceptance Prediction — Tuần 4: Model DL sai ở đâu nhiều nhất?

Dự án	Rider Acceptance (Matching & Dispatch)	Phần	Tiếp nối Tuần 2 (phân tích lỗi theo segment) — áp dụng cho 3 model DL của Tuần 3
Mục tiêu	Xác định 3 model DL (FT-Transformer V2, MambaTab V2, MLP V2) sai nhiều nhất ở segment/feature nào — so sánh chéo với GBDT (XGBoost V2) để biết DL thua GBDT chung chung hay chỉ ở vài chỗ cụ thể	Test set	13/07/2026, n=8.609 đơn post-dispatch (giống hệt W2/W3)
Model phân tích	XGBoost V2, MLP V2, FT-Transformer V2, MambaTab V2 — đều dùng hyperparameter Optuna-tuned từ <code>artifacts/metrics_*_v2.json</code> , tối ưu theo Cancel PR-AUC (chuẩn W3)	Tái lập	<code>python3</code> <code>Baseline/run_error_analysis_dl.p</code> rồi <code>python3</code> <code>Baseline/segment_analysis_dl.py</code>

0. Phạm vi & phương pháp

Tiếp nối đúng phương pháp báo cáo Tuần 2 (`reports/part1_shap.md`) — segment theo **khách mới/quen, giờ cao điểm/thường, ETA dài/ngắn** (cùng định nghĩa `error_analysis.py`) — nhưng lần này so sánh **4 MODEL CLASS** trên CÙNG 1 test set, thay vì 2 architecture của 1 model. Thêm **XGBoost V2** (GBDT thắng ở W3 mục 3b) làm đối chứng, để tách 2 câu hỏi:

- 1. **3 model DL có yếu ở CÙNG 1 kiểu phân khúc không, hay mỗi model yếu 1 kiểu khác nhau?**
- 2. **DL thua GBDT ở MỌI nơi (đơn nào GBDT cũng nhỉnh hơn), hay chỉ thua ở 1 nhóm đơn cụ thể — và nhóm đó có đặc điểm gì?**

Ngoài bảng segment + soi trực tiếp (median feature FN vs TP, giống W2), thêm **SHAP/feature-attribution cho cả 3 model DL** — dùng `shap.Explainer(algorithm="permutation")` trên raw feature space (không phải TreeExplainer như LightGBM ở W2, vì DL không phải cây) — để so xem feature nào "gây lỗi" nhiều nhất cho từng model, có trùng với GBDT không.

Giới hạn cần lưu ý trước khi đọc: SHAP của 3 model DL tính trên **mẫu 200 đơn FN** (không phải toàn bộ 8.609 — Permutation explainer quá chậm cho quy mô đó với 3 model), background 40 dòng từ train. So sánh XGBoost dùng TreeExplainer (nhanh,

chính xác, không cần sample) trên đúng cỡ mẫu FN tương ứng để công bằng khi so sánh. **Độ lớn SHAP value giữa Permutation explainer (DL) và TreeExplainer (XGBoost) không cùng đơn vị** — chỉ so sánh được THỨ HẠNG feature trong từng model, không so trực tiếp con số tuyệt đối giữa DL và XGBoost.

```
In [1]: %matplotlib inline
import json
import os

import numpy as np
import pandas as pd

pd.set_option('display.width', 160)
pd.set_option('display.max_columns', 20)

ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
OUT = os.path.join(ROOT, 'artifacts', 'dl_error_analysis')

labels = json.load(open(os.path.join(OUT, 'model_labels.json')))
labels
```

```
Out[1]: {'XGBoost': 'xgboost V2 (Optuna-tuned)',
'MLP': 'MLP V2 (Optuna-tuned)',
'FT-Transformer': 'FT-Transformer V2 (Optuna-tuned)',
'MambaTab': 'MambaTab V2 (Optuna-tuned)'}
```

1. Model đang phân tích (hyperparameter V2 đã tune, theo Cancel PR-AUC)

```
In [2]: pred_df = pd.read_csv(os.path.join(OUT, 'predictions_test.csv'), index_col=0)
MODEL_COLS = ['XGBoost', 'MLP', 'FT-Transformer', 'MambaTab']
print(f"test set: {len(pred_df):,} đơn - base rate accept {pred_df['y_accepted'].mean():.2f}")
pd.Series(labels)
```

test set: 8,609 đơn - base rate accept 0.9080

```
Out[2]: XGBoost          xgboost V2 (Optuna-tuned)
MLP                MLP V2 (Optuna-tuned)
FT-Transformer    FT-Transformer V2 (Optuna-tuned)
MambaTab          MambaTab V2 (Optuna-tuned)
dtype: object
```

2. ROC-AUC / Cancel PR-AUC theo segment — 4 model song song

Cùng định nghĩa segment với `error_analysis.py` (W1/W2): khách mới = `cust_orders_30d` NaN hoặc 0; giờ cao điểm = {7,8,9,17,18,19}h; ETA dài = >600s.

```
In [3]: seg_df = pd.read_csv(os.path.join(OUT, 'segment_metrics.csv'))
pivot_roc = seg_df.pivot(index='segment', columns='model', values='roc_auc')
pivot_prc = seg_df.pivot(index='segment', columns='model', values='pr_auc_cancel')
```

```
order = ['Toàn bộ post-dispatch', 'Khách mới', 'Khách quen', 'Giờ cao điểm',
        'ETA dài (>600s)', 'ETA ngắn (≤600s)']
print("ROC-AUC theo segment:")
display(pivot_roc.loc[order].round(4))
print("\nCancel PR-AUC theo segment (metric quyết định chính):")
display(pivot_prc.loc[order].round(4))
```

ROC-AUC theo segment:

model	XGBoost	MLP	FT-Transformer	MambaTab
segment				
Toàn bộ post-dispatch	0.7488	0.7415	0.7366	0.7383
Khách mới	0.7579	0.7300	0.7317	0.7254
Khách quen	0.7481	0.7427	0.7373	0.7394
Giờ cao điểm	0.7565	0.7537	0.7518	0.7524
Giờ thường	0.7439	0.7338	0.7267	0.7290
ETA dài (>600s)	0.7778	0.7436	0.8034	0.7650
ETA ngắn (≤600s)	0.7435	0.7396	0.7313	0.7364

Cancel PR-AUC theo segment (metric quyết định chính):

model	XGBoost	MLP	FT-Transformer	MambaTab
segment				
Toàn bộ post-dispatch	0.2704	0.2441	0.2400	0.2454
Khách mới	0.3218	0.2502	0.2606	0.2280
Khách quen	0.2669	0.2456	0.2403	0.2483
Giờ cao điểm	0.3020	0.2891	0.2816	0.2810
Giờ thường	0.2513	0.2182	0.2151	0.2261
ETA dài (>600s)	0.4624	0.4144	0.4148	0.3197
ETA ngắn (≤600s)	0.2623	0.2440	0.2372	0.2459

Độc bảng: ETA dài (>600s) chỉ có n=45 — quá nhỏ để tin cậy thống kê (giống lưu ý ở W2), chỉ mang tính tham khảo.

3. Segment yếu nhất mỗi model

```
In [4]: worst = (seg_df[seg_df.segment != 'Toàn bộ post-dispatch']
               .sort_values('pr_auc_cancel').groupby('model').first()[['segment',
               .loc[MODEL_COLS]])
               worst
```

Out [4]:

	segment	n	pr_auc_cancel
model			
XGBoost	Giờ thường	5337	0.2513
MLP	Giờ thường	5337	0.2182
FT-Transformer	Giờ thường	5337	0.2151
MambaTab	Giờ thường	5337	0.2261

Phát hiện chính #1 — cả 4 model (3 DL + GBDT) đều yếu nhất ở CÙNG 1 segment:

"Giờ thường" (không phải giờ cao điểm như trực giác thường nghĩ). Đây không phải đặc thù của kiến trúc DL — GBDT cũng yếu nhất ở đúng segment này. Khác biệt là **mức độ**: XGBoost vẫn giữ Cancel PR-AUC 0,2513 ở "Giờ thường", trong khi FT-Transformer tụt xuống 0,2151 (thấp hơn ~14%) — GBDT không tránh được điểm yếu này, chỉ chịu đựng tốt hơn.

Một phát hiện phụ đáng chú ý: **FT-Transformer bắt được 0 đơn huỷ ở segment**

"Khách mới" (`recall_cancel = 0,0000`, xem bảng mục 2) — tệ hơn cả 2 model DL còn lại (MLP/MambaTab đều còn bắt được 1/81 đơn, recall 0,0123). Tái khẳng định phát hiện "khách mới yếu nhất" từ W2, nhưng lần này FT-Transformer là model bỏ sót HOÀN TOÀN nhóm này.

4. FN overlap — 3 model DL có sai trên CÙNG những đơn hay không?

```
In [5]: overlap = json.load(open(os.path.join(OUT, 'fn_overlap_summary.json')))
pd.Series(overlap['fn_counts'])
```

```
Out[5]: XGBoost      765
MLP          779
FT-Transformer  785
MambaTab     769
dtype: int64
```

```
In [6]: n_huy = int(pred_df['y_accept'].eq(0).sum())
print(f"Tổng số đơn huỷ thật trong test: {n_huy}")
print(f"Union FN (>=1 trong 3 DL sai):      {overlap['dl_fn_union']:4d} ({n_huy}")
print(f"Intersection FN (CẢ 3 DL đều sai):  {overlap['dl_fn_intersection']}")
print(f"XGBoost FN:                        {overlap['fn_counts']['XGBoost']}")
print(f"'DL-only-wrong' (cả 3 DL sai, XGBoost đúng): {overlap['dl_only_wrong']}
```

```
Tổng số đơn huỷ thật trong test: 792
Union FN (>=1 trong 3 DL sai):      787 (99.4%)
Intersection FN (CẢ 3 DL đều sai):  766 (96.7%)
XGBoost FN:                        765 (96.6%)
'DL-only-wrong' (cả 3 DL sai, XGBoost đúng):  11
```

Phát hiện chính #2 — 3 model DL sai gần như CÙNG 1 tập đơn, không phải 3 kiểu lỗi độc lập. Intersection (cả 3 DL cùng sai) = 766/787 union = **97,3%** — nghĩa là hầu như bất kỳ đơn nào 1 model DL bỏ lỡ, 2 model DL còn lại cũng bỏ lỡ theo. Pairwise Jaccard giữa từng cặp DL đều $\geq 97,5\%$ (xem log `segment_analysis_dl.py`). **Đây là bằng chứng khá mạnh rằng 3 kiến trúc DL (token-per-feature Transformer, selective-scan SSM, MLP thuần) đều bị giới hạn bởi CÙNG 1 nguyên nhân gốc — nhiều khả năng là tín hiệu/feature hiện có không đủ tách nhóm khó này, chứ không phải hạn chế riêng của 1 kiến trúc nào.**

So với XGBoost: XGBoost's FN set (765) gần như là TẬP CON của intersection DL (766) — chỉ **11 đơn** mà cả 3 DL đều sai nhưng XGBoost lại đúng. Ngược lại (GBDT sai, DL đúng) không xảy ra đáng kể ở mức tổng quát này — tức **GBDT không "bù" được cho DL ở phần lớn các đơn khó, GBDT chỉ đơn giản là bắt đúng NHIỀU HƠN trên CÙNG một quần thể đơn khó** (base rate huỷ quá thấp $\sim 9\%$, threshold 0,5 quá cao — vấn đề đã nêu ở W2), không phải có 1 nhóm đơn "DL nhìn ra nhưng GBDT bỏ lỡ".

5. Soi trực tiếp: 11 đơn "DL-only-wrong" so với nhóm DL bắt đúng

⚠ **n=11 vs n=5 — mẫu RẤT nhỏ, chỉ mang tính minh họa/gợi ý hướng, không phải kết luận thống kê chắc chắn** (giống tinh thần lưu ý "n=6" ở phần FP của W2).

```
In [7]: num_comp = pd.read_csv(os.path.join(OUT, 'dl_only_wrong_vs_correct_numeric.c
num_comp.round(2)
```

Out [7]:

	DL-only-wrong (median)	DL-all-correct TP (median)	% lệch
pickup_distance_km	2.04	0.35	483.34
cust_completion_rate_30d	0.65	0.19	236.82
eta_seconds	446.07	141.03	216.29
hour_of_day	17.00	9.00	88.89
total_fee	41000.00	335000.00	-87.76
trip_distance_km	3.61	28.55	-87.36
cust_cancel_rate_30d	0.25	0.79	-68.33
cust_orders_30d	25.00	53.00	-52.83
dist_hotspot_km	2.53	4.90	-48.39
fee_per_km	15424.61	11124.25	38.66
re_dispatch_counts	2.00	3.00	-33.33
dist_airport_km	26.75	24.80	7.86
dist_center_km	11.62	12.54	-7.34
surge_multiplier	1.00	1.00	0.00
is_weekend	0.00	0.00	NaN
is_schedule_order	0.00	0.00	NaN
is_rainy_3h	1.00	0.00	NaN

Nhóm "DL-only-wrong" (11 đơn cả 3 DL đều đoán sai, XGBoost đoán đúng) có xu hướng: **pickup_distance_km** cao hơn hẳn (+483%, tài xế phải chạy xa hơn nhiều để đón), **eta_seconds** dài hơn (+216%), **giờ đặt muộn hơn** (17h vs 9h) — NHƯNG **total_fee** thấp hơn nhiều (-88%) và **trip_distance_km** ngắn hơn nhiều (-87%) so với nhóm cả 3 DL bắt đúng. Diễn giải định tính: đây có vẻ là nhóm "**chuyến ngắn, giá rẻ, nhưng tài xế phải đón xa**" — một pattern khá cụ thể mà GBDT (nhờ khả năng cắt ngưỡng phi tuyến dứt khoát trên từng feature) nắm bắt tốt hơn 3 kiến trúc DL trong mẫu nhỏ này.

```
In [8]: cat_comp = pd.read_csv(os.path.join(OUT, 'dl_only_wrong_vs_correct_categoricals.csv'))
cat_comp.round(1)
```

Out [8]:

	feature	value	wrong_%	correct_%	diff_pp
0	travel_mode	car	54.5	100.0	-45.5
1	vertical_name	Taxi	54.5	100.0	-45.5
2	travel_mode	motorcycle	45.5	0.0	45.5
3	vertical_name	Express	36.4	0.0	36.4
4	payment_method	momo	9.1	20.0	-10.9
5	vertical_name	Bike	9.1	0.0	9.1
6	payment_method	zalo	9.1	0.0	9.1
7	payment_method	international_card	9.1	0.0	9.1
8	payment_method	cash	72.7	80.0	-7.3
9	payment_method	closed_wallet	0.0	0.0	0.0
10	payment_method	vsf	0.0	0.0	0.0
11	payment_method	vnpay_pos	0.0	0.0	0.0
12	payment_method	shopee_pay	0.0	0.0	0.0
13	payment_method	msb_qr	0.0	0.0	0.0
14	payment_method	onipay_auto_debit	0.0	0.0	0.0
15	payment_method	physical_business_card	0.0	0.0	0.0
16	payment_method	partner_pay	0.0	0.0	0.0
17	payment_method	hdb_qr	0.0	0.0	0.0
18	payment_method	business_card	0.0	0.0	0.0
19	payment_method	viettel_money	0.0	0.0	0.0
20	vertical_name	Food	0.0	0.0	0.0
21	travel_mode	truck	0.0	0.0	0.0
22	payment_method	qr	0.0	0.0	0.0

6. SHAP / feature-attribution — feature nào "gây lỗi" nhiều nhất mỗi model?

Tính trên mẫu FN (huỷ, đoán sai) của từng model riêng (~200 đơn/model, xem mục 0 về giới hạn phương pháp). Xếp theo mean(|SHAP value|) — feature ảnh hưởng lớn nhất tới quyết định (đúng hoặc sai) trên đúng nhóm đơn model đang sai.

```
In [9]: shap_tables = {}
for name in ['xgboost', 'mlp', 'ft_transformer', 'mambatab']:
    s = pd.read_csv(os.path.join(OUT, f'shap_meanabs_{name}.csv'), index_col=1)
```

```

shap_tables[name] = s

rank_df = pd.DataFrame({
    'XGBoost (TreeExplainer)': shap_tables['xgboost'].rank(ascending=False),
    'MLP (Permutation)': shap_tables['mlp'].rank(ascending=False).astype(int),
    'FT-Transformer (Permutation)': shap_tables['ft_transformer'].rank(ascending=False),
    'MambaTab (Permutation)': shap_tables['mambatab'].rank(ascending=False)
})
rank_df['rank trung bình (3 DL)'] = rank_df[['MLP (Permutation)', 'FT-Transformer (Permutation)', 'MambaTab (Permutation)']].mean(axis=1)
rank_df.sort_values('rank trung bình (3 DL)').head(10)

```

Out[9]:

	XGBoost (TreeExplainer)	MLP (Permutation)	FT- Transformer (Permutation)	MambaTab (Permutation)
pickup_distance_km	1	2	1	1
cust_cancel_rate_30d	2	3	3	4
eta_seconds	3	4	2	5
travel_mode	17	1	9	2
vertical_name	6	5	6	7
trip_distance_km	4	7	4	9
re_dispatch_counts	13	6	8	6
fee_per_km	7	9	5	8
total_fee	14	8	11	3
cust_orders_30d	8	10	10	10

Phát hiện chính #3 — cả 4 model class (kể cả GBDT) hội tụ về CÙNG 1 nhóm feature "khó" khi giải thích lỗi: `pickup_distance_km`, `cust_cancel_rate_30d`, `eta_seconds` nằm trong top-4 của cả 4 model (xem hạng trung bình). `travel_mode` / `vertical_name` cũng lặp lại ở top của cả 3 model DL (MLP hạng 1, MambaTab hạng 2, FT-Transformer thấp hơn nhưng vẫn top-10).

Kết hợp với phát hiện #2 (3 DL sai trên gần như CÙNG 1 tập đơn), bức tranh nhất quán: **độ khó không nằm ở việc model nào "nhìn thấy" feature nào — cả 4 model đều dựa vào đúng những feature có tín hiệu mạnh nhất (`pickup_distance_km`, `cust_cancel_rate_30d`, `eta_seconds`). Vấn đề là những feature này KHÔNG đủ tách được nhóm đơn khó (đơn huỷ "bất ngờ", không theo pattern rõ) — một giới hạn của DỮ LIỆU hiện có, không phải giới hạn riêng của bất kỳ kiến trúc DL nào.**

Bảng chi tiết top-8 mỗi model (giá trị mean(|SHAP|), không so được tuyệt đối giữa 2 loại explainer — chỉ đọc thứ tự trong từng cột):


```
In [10]: top8 = pd.DataFrame({
    'XGBoost': shap_tables['xgboost'].head(8).index,
    'MLP': shap_tables['mlp'].head(8).index,
    'FT-Transformer': shap_tables['ft_transformer'].head(8).index,
    'MambaTab': shap_tables['mambatab'].head(8).index,
  })
top8.index = [f'#{i+1}' for i in range(8)]
top8
```

Out[10]:

	XGBoost	MLP	FT-Transformer	Mar
#1	pickup_distance_km	travel_mode	pickup_distance_km	pickup_distai
#2	cust_cancel_rate_30d	pickup_distance_km	eta_seconds	trave
#3	eta_seconds	cust_cancel_rate_30d	cust_cancel_rate_30d	tc
#4	trip_distance_km	eta_seconds	trip_distance_km	cust_cancel_ra
#5	hour_of_day	vertical_name	fee_per_km	eta_s
#6	vertical_name	re_dispatch_counts	vertical_name	re_dispatch_
#7	fee_per_km	trip_distance_km	cust_completion_rate_30d	vertica
#8	cust_orders_30d	total_fee	re_dispatch_counts	fee_

7. Kết luận chung

1. **3 model DL không có 3 kiểu lỗi khác nhau — chúng chia sẻ gần như CÙNG 1 điểm mù** (97,3% FN trùng nhau, cùng segment yếu nhất "Giờ thường", cùng top feature SHAP). Nếu mục tiêu W4 là cải thiện DL, hướng đúng là tìm **feature/dữ liệu mới** giúp tách nhóm đốn khó này (giống khuyến nghị W2), không phải thử thêm kiến trúc DL khác — 3 kiến trúc rất khác nhau (Transformer/SSM/MLP) đã hội tụ về cùng 1 giới hạn, gợi ý đây là giới hạn dữ liệu chứ không phải giới hạn kiến trúc.
2. **GBDT (XGBoost) không "nhìn ra" một nhóm đốn hoàn toàn khác với DL — nó chỉ giỏi hơn TRÊN CÙNG quần thể đốn khó** (chỉ 11/787 đốn là "DL-only-wrong", XGBoost gần như tập con của DL về mặt sai sót). Điều này củng cố thêm phát hiện W3 (bootstrap CI): GBDT thắng DL một cách nhất quán, không phải nhờ bù trừ ở 1 nhóm đốn riêng biệt.
3. **"Khách mới" và "Giờ thường" tiếp tục là 2 segment yếu nhất** — nhất quán với W2 (baseline LightGBM v1) lẫn W4 (4 model V2 hiện tại). FT-Transformer đặc biệt tệ ở "Khách mới" (recall_cancel = 0). Đây là bằng chứng lặp lại nhiều lần (không phải ngẫu nhiên 1 lần đo) rằng vấn đề khách mới là **cold-start dữ liệu**, không phải model chưa đủ mạnh — nhất quán với kết luận W2.
4. **11-mẫu "DL-only-wrong" gợi ý (KHÔNG khẳng định)** một pattern "chuyển ngắn/giá rẻ nhưng tài xế đốn xa" mà GBDT nắm được còn DL thì không — đáng thử lại với mẫu lớn hơn (chờ thêm dữ liệu hoặc gộp nhiều ngày test) trước khi kết luận chắc.

8. Addendum (12/08/2026) — Thử cải thiện FT-Transformer để đánh bại XGBoost

Sau khi mục 6-7 chỉ ra 3 model DL chia sẻ chung 1 điểm mù (không phải hạn chế riêng kiến trúc nào), câu hỏi tự nhiên tiếp theo: **có cách nào cải thiện FT-Transformer đủ để đánh bại XGBoost V2 (Cancel PR-AUC 0,2704) không?** Thử nghiệm 4 hướng, **cô lập từng biến thay đổi 1 lần** để biết chính xác cái nào giúp/hại — không gộp chung rồi đoán mò.

Tái lập: `python3 Baseline/train_ft_transformer_v3.py --ablation {full,ple_only,leaf_only,none} --loss {bce,focal}`, `python3 Baseline/train_ft_transformer_v3_calibrated.py`, `python3 Baseline/train_dl_focal_comparison.py`.

8.1 · Kiến trúc: Piecewise-linear numeric encoding + GBDT leaf-embedding hybrid


```

{'Model': 'FT-Transformer V2', 'Loss': 'Focal', 'ROC-AUC': 0.7441, 'Cancel PR-AUC': 0.2508, 'ECE': 0.3543, 'Δ Cancel PR-AUC vs BCE': 0.0108}
{'Model': 'MLP V2', 'Loss': 'BCE', 'ROC-AUC': 0.7415, 'Cancel PR-AUC': 0.2460, 'ECE': 0.0067, 'Δ Cancel PR-AUC vs BCE': 0.0000}
{'Model': 'MLP V2', 'Loss': 'Focal', 'ROC-AUC': 0.7377, 'Cancel PR-AUC': 0.2400, 'ECE': 0.3667, 'Δ Cancel PR-AUC vs BCE': -0.0060}
{'Model': 'MambaTab V2', 'Loss': 'BCE', 'ROC-AUC': 0.7384, 'Cancel PR-AUC': 0.2475, 'ECE': 0.0154, 'Δ Cancel PR-AUC vs BCE': 0.0000}
{'Model': 'MambaTab V2', 'Loss': 'Focal', 'ROC-AUC': 0.7368, 'Cancel PR-AUC': 0.2434, 'ECE': 0.3590, 'Δ Cancel PR-AUC vs BCE': -0.0041}
]
focal_df = pd.DataFrame(focal_rows)
focal_df['Δ Cancel PR-AUC vs BCE'] = focal_df.groupby('Model')['Cancel PR-AUC'] - focal_df['ECE']
focal_df

```

Out [12]:

	Model	Loss	ROC-AUC	Cancel PR-AUC	ECE	Δ Cancel PR-AUC vs BCE
0	FT-Transformer V2	BCE	0.7366	0.2400	0.0060	0.0000
1	FT-Transformer V2	Focal	0.7441	0.2508	0.3543	0.0108
2	MLP V2	BCE	0.7415	0.2460	0.0067	0.0000
3	MLP V2	Focal	0.7377	0.2400	0.3667	-0.0060
4	MambaTab V2	BCE	0.7384	0.2475	0.0154	0.0000
5	MambaTab V2	Focal	0.7368	0.2434	0.3590	-0.0041

Phát hiện quan trọng — Focal loss KHÔNG phải "công thức chung cho DL trên dữ liệu imbalanced" như có thể nghĩ ban đầu:

- **FT-Transformer:** Focal giúp thật — ROC-AUC +0,0075, Cancel PR-AUC **+0,0108**. Recall lớp huỷ nhảy từ ~0,01 lên ~0,68.
- **MLP:** Focal làm HẠI cả 2 mặt — Cancel PR-AUC **-0,0060**, ECE tăng x55 mà không đổi được ranking tốt hơn để bù.
- **MambaTab:** tương tự MLP — Cancel PR-AUC **-0,0041**, cùng kiểu ECE vớ vô ích.

Tác dụng của Focal loss có vẻ **phụ thuộc kiến trúc**, không phải đặc tính chung của "dữ liệu imbalanced + DL". Giả thuyết hợp lý nhất: cơ chế self-attention của FT-Transformer (cho phép model "chú ý" chọn lọc vào 1 số token/feature nhất định mỗi dự đoán) tận dụng được gradient tái trọng số của Focal tốt hơn hẳn kiến trúc không có attention (MLP thuần, MambaTab dùng selective-scan nhưng không phải attention thực sự) — nhưng đây là suy đoán, chưa kiểm chứng thêm.

8.3 · Calibration bắt buộc sau Focal Loss (áp cho FT-Transformer, model duy nhất Focal thật sự giúp)

```

In [13]:
calib_rows = [
    {'Bước': 'Trước calib (raw, ple_only + Focal, threshold=0.5)', 'ROC-AUC': 0.7441, 'Cancel PR-AUC': 0.2508, 'ECE': 0.3543, 'Δ Cancel PR-AUC vs BCE': 0.0108},
    {'Bước': 'Sau calib (isotonic) + threshold=0.84 chọn lại', 'ROC-AUC': 0.7415, 'Cancel PR-AUC': 0.2460, 'ECE': 0.0067, 'Δ Cancel PR-AUC vs BCE': 0.0000}
]
pd.DataFrame(calib_rows)

```

Out [13]:

	Bước	ROC-AUC	Cancel PR-AUC	ECE	Recall huỷ
0	Trước calib (raw, ple_only + Focal, threshold=...	0.7450	0.2517	0.3543	0.6831
1	Sau calib (isotonic) + threshold=0.84 chọn lại	0.7435	0.2375	0.0041	0.4571

Cùng pattern đã ghi nhận ở W3 mục 6-7 (imbalance luôn đánh đổi calibration, luôn cần bước calibration riêng sau): fit 3 phương pháp (spline/isotonic/platt) trên tập `calib`, chọn theo ECE đo trên `valid` (không dùng test) — isotonic thắng (ECE valid 0,0068). Sau calibration, ECE giảm từ 0,3543 xuống **0,0041** (dùng được thật), nhưng **Cancel PR-AUC giảm nhẹ theo (0,2517→0,2375)** — hiện tượng isotonic regression tạo tie ở biên, làm lệch nhẹ thứ hạng dù về lý thuyết là hàm đơn điệu (đã ghi nhận tương tự ở W3).

8.4 · Bảng tổng hợp toàn bộ 8 biến thể đã thử

```
In [14]: final_rows = [
    {'Biến thể': 'XGBoost V2 (đối chứng, GBDT thắng W3)', 'ROC-AUC': 0.7488,
    {'Biến thể': 'FT-Transformer V2 gốc (BCE)', 'ROC-AUC': 0.7366, 'Cancel PR-AUC': 0.2400,
    {'Biến thể': 'ple_only (BCE)', 'ROC-AUC': 0.7424, 'Cancel PR-AUC': 0.2437,
    {'Biến thể': 'leaf_only (BCE)', 'ROC-AUC': 0.7367, 'Cancel PR-AUC': 0.2402,
    {'Biến thể': 'full = ple+leaf (BCE)', 'ROC-AUC': 0.7341, 'Cancel PR-AUC': 0.2366,
    {'Biến thể': 'V2 gốc + Focal (raw)', 'ROC-AUC': 0.7441, 'Cancel PR-AUC': 0.2508,
    {'Biến thể': 'ple_only + Focal (raw)', 'ROC-AUC': 0.7450, 'Cancel PR-AUC': 0.2517,
    {'Biến thể': 'ple_only + Focal + Calibrated', 'ROC-AUC': 0.7435, 'Cancel PR-AUC': 0.2375,
    ]
result = pd.DataFrame(final_rows).sort_values('Cancel PR-AUC', ascending=False)
result
```

Out [14]:

	Biến thể	ROC-AUC	Cancel PR-AUC	Dùng được ngay?
0	XGBoost V2 (đối chứng, GBDT thắng W3)	0.7488	0.2704	✅
6	ple_only + Focal (raw)	0.7450	0.2517	❌ ECE=0,35
5	V2 gốc + Focal (raw)	0.7441	0.2508	❌ ECE=0,35
2	ple_only (BCE)	0.7424	0.2437	✅
3	leaf_only (BCE)	0.7367	0.2402	✅
1	FT-Transformer V2 gốc (BCE)	0.7366	0.2400	✅
7	ple_only + Focal + Calibrated	0.7435	0.2375	✅
4	full = ple+leaf (BCE)	0.7341	0.2366	✅ (nhưng tệ hơn gốc)

8.5 · Kết luận addendum

1. **Không biến thể nào đánh bại được XGBoost V2 (0,2704)** — biến thể tốt nhất DÙNG ĐƯỢC NGAY (đã calibrate) chỉ đạt 0,2472-0,2517 ở dạng raw hoặc 0,2375 sau calib, đều thấp hơn XGBoost 0,02-0,03. Kể cả biến thể raw tốt nhất (`ple_only + Focal` , 0,2517, chưa dùng được vì ECE vỡ) vẫn thua XGBoost 0,019.
2. **Focal loss là đòn bẩy thật sự duy nhất tìm được** (+0,0108 Cancel PR-AUC cho FT-Transformer) — nhưng **chỉ hiệu quả với FT-Transformer**, làm HẠI cả MLP lẫn MambaTab. Không có công thức chung "cứ imbalance thì dùng Focal" cho mọi kiến trúc DL.
3. **PLE encoding đóng góp thật nhưng nhỏ và không cộng dồn tuyến tính** với Focal (`ple_only+Focal` chỉ nhỉnh hơn V2-gốc+Focal 0,0009, trong biên độ nhiễu) — phần lớn cải thiện đến từ Focal, không phải PLE.
4. **Leaf-embedding hybrid nên bỏ hẳn** — trung tính khi đứng riêng, có hại khi cộng với PLE (overfit).
5. Cùng cố thêm kết luận mục 7: **giới hạn nằm ở dữ liệu, không phải kỹ thuật huấn luyện/kiến trúc còn thiếu** — đã thử khá đầy đủ (encoding mới, hybrid với GBDT, loss imbalance-aware, calibration đúng chuẩn) mà khoảng cách với XGBoost vẫn không thu hẹp được về 0.